

SMART: A Selective Marker-guided Adaptive Recursive Transformer with Learned Gene-Graph Routing for Single-Cell and Multi-Omics Classification

Anonymous Submission

Abstract

Transformer models for single-cell and multi-omics classification treat thousands of genes as equally important tokens and stack many independent layers, making them parameter-heavy and leaving *which* genes deserve computation entirely to data. We present **SMART** (Selective Marker-guided Adaptive Recursive Transformer), which (i) learns which genes are *markers* via a cross-attention router and represents each sample by only its $M \ll N$ markers, cutting attention from $\mathcal{O}(N^2)$ to $\mathcal{O}(M^2)$; (ii) processes them with a *single* weight-shared block applied recursively, a Mixture-of-Recursions router giving each token its own adaptive depth; and (iii) extends to bulk multi-omics via fixed *Reactome pathway* tokens pooling mutation, copy-number and expression. Across 11 datasets (8 single-cell, 3 multi-omics) our controlled *none / random / fixed-biology / learned* study finds the *learned* gene-graph is the decisive positive (+11.7 points to 70.8% single-cell macro-F1, above a degree-matched random graph 57.5%), and a confound factorial isolates the gain as input *smoothing*, not depth routing. We surface honest negatives: a *fixed* hand-built prior helps neither accuracy nor calibration. Efficiency is architectural: a $4\times$ parameter reduction at $K=4$, $\sim 38\%$ fewer recursion FLOPs at matched accuracy, and a few dozen tokens recovering most full-gene accuracy. Classical linear baselines remain strong on the engineered single-cell features, while on the raw multi-omics cohorts SMART’s best multi-modal model is the strongest method; it also matches a far larger pretrained foundation model within noise. The whole pipeline and this paper regenerate from a single command.

1 Introduction

Single-cell RNA sequencing now profiles the expression of thousands of genes across millions of cells, and a wave of transformer *foundation models*, scGPT (Cui et al. 2024), Geneformer (Theodoris et al. 2023), scBERT (Yang et al. 2022) and scFoundation (Hao et al. 2024), has adapted the architecture of (Vaswani et al. 2017) to this modality. These models are powerful but inherit two costly habits from language transformers. First, they treat *every* gene as an equally important token, so a housekeeping gene and a lineage-defining marker get identical computational budgets. Second,

they stack many *independent* layers, so parameters grow linearly with depth. The result is models with tens to hundreds of millions of parameters (Hao et al. 2024) whose self-attention scales quadratically in the number of genes, and whose efficiency is usually recovered only afterward through pruning or distillation.

We take a different stance: *for gene expression, the data and the known biology together tell us where to spend computation*. Decades of single-cell biology rest on two facts. A small set of *marker genes* is sufficient to discriminate cell types (Ianevski, Giri, and Aittokallio 2022; Franzén, Gan, and Björkegren 2019; Hu et al. 2023); and gene co-expression and regulatory networks are approximately scale-free, so a few high-degree *hub* genes exert outsized influence. If a model could decide, during training, which genes are markers, and could be told, before training, which genes are network hubs, it could grant those genes dedicated capacity and let everything else share parameters. This makes parameter efficiency an *architectural* property and lets biology shape the *routing decision* rather than merely validate it afterward.

We realise this in **SMART**, a recursive marker-guided transformer with three coupled components and one biological prior. (1) A cross-attention *marker router*: M learnable queries attend over all N genes with a temperature-annealed softmax (Set-Transformer / Perceiver-style (Jang, Gu, and Poole 2017; Baln, Abid, and Zou 2019)), so the model learns *which* genes are markers end-to-end while gradients reach every gene. (2) *Marker-driven compression*: each cell is represented by only its $M \ll N$ markers, cutting attention from $\mathcal{O}(N^2)$ to $\mathcal{O}(M^2)$. (3) A *recursive shared block*: one transformer block applied K times in the spirit of Universal Transformers (Dehghani et al. 2019), ALBERT (Lan et al. 2020) and Mixture-of-Recursions (Bae et al. 2025), with an expert-choice depth router that gives each gene an adaptive recursion depth. On top of this, a *biology-informed router* folds a gene-gene interaction graph into that depth decision. Our central finding concerns *how* that graph should be formed: a *fixed* hand-built graph (genomap co-expression (Islam and Xing 2023) or Reactome centrality) does not beat a degree-matched random-graph control, whereas a *learned*, data-driven graph trained end-to-end does – so biology helps routing only when the graph is learned rather than imposed a priori.

We make the following contributions:

- **SMART**, a transformer for genomic classification that co-designs token selection, token compression, and parameter-shared recursion end-to-end, with each token’s recursion depth as an intrinsic compute-allocation score. The token interface is modality-general: *learned marker genes* for single-cell expression and fixed *Reactome pathway tokens* (pooling mutation, copy-number and expression) for bulk multi-omics.
- A **controlled none / random-graph / fixed-biology / learned study** of the gene-graph prior showing the **learned** graph is the decisive positive (+11.7 points over a no-prior router, to 70.8% single-cell macro-F1), while a fixed hand-built prior does not separate from a degree-matched random-graph control; a confound factorial further isolates the gain as input *smoothing*, not depth routing (Tables 2, 3).
- **Efficiency that is architectural**: weight sharing gives a $4\times$ parameter reduction at $K=4$, adaptive expert-choice recursion cuts $\sim 38\%$ of recursion FLOPs at matched accuracy, and a few dozen marker tokens recover most full-gene accuracy (Tables 9, 10, 7).
- An **honest evaluation** on all 11 datasets (8 single-cell, 3 multi-omics), multi-seed, surfacing negatives (fixed priors help neither accuracy nor calibration) and comparing to strong classical and foundation-model baselines; the full pipeline and paper regenerate from a single command.

2 Related Work

Transformer foundation models for single-cell omics.

Geneformer (Theodoris et al. 2023) and scBERT (Yang et al. 2022) adapt masked-language-model pretraining to single-cell transcriptomes; scGPT (Cui et al. 2024) scales generative pretraining to 33M cells; scFoundation (Hao et al. 2024) trains a 100M-parameter model over $\sim 20,000$ genes; CellPLM (Wen et al. 2024) and Cell2Sentence (Levine et al. 2024) push cell- and language-level pretraining further. Earlier deep generative approaches such as scVI (Lopez et al. 2018) established probabilistic latent representations. genomap (Islam and Xing 2023) instead reshapes the gene vector into an image via an optimal-transport layout built from a gene-gene *interaction* matrix, and uses a small CNN (genoNet) for cell recognition; we reuse precisely that interaction identification, but as a routing prior rather than an image layout. All of these treat genes uniformly or select them in a separate stage; none make marker-driven sparsity an architectural prior or let a gene-interaction graph shape adaptive computation.

Parameter-efficient and recursive transformers. Tying weights across depth was shown to retain representational power by Universal Transformers (Dehghani et al. 2019) and to shrink models in ALBERT (Lan et al. 2020). Mixture-of-Recursions (Bae et al. 2025) unifies weight sharing with token-level adaptive depth, and Mixture-of-Depths (Raposo et al. 2024) routes tokens through variable numbers of layers, both echoing sparsely-gated mixtures of experts (Shazeer et al. 2017) and adaptive computation time (Graves 2016). Efficient-attention methods, Linformer (Wang et al. 2020),

Performer (Choromanski et al. 2021) and Nyströmformer (Xiong et al. 2021), reduce the quadratic cost generically, and recursive weight sharing has been pushed further in vision and restoration transformers (Shen, Liu, and Xing 2022; Jaber and Jaber 2026). We borrow the weight-sharing mechanism but make the token set biologically structured and the routing biologically primed, so our $\mathcal{O}(M^2)$ saving and our prior are complementary to these methods.

Markers, networks, and biological priors. Marker-based annotation tools such as scType (Ianevski, Giri, and Aittokallio 2022), Cell-ID (Cortal et al. 2021) and SingleR (Aran et al. 2019), and curated databases including PanglaoDB (Franzén, Gan, and Björkegren 2019) and CellMarker 2.0 (Hu et al. 2023), encode the principle that few genes carry most discriminative signal. Pathway-informed models such as the graph transformer PATH (Howlader, Islam, and Le 2026) build structure from Reactome (Gillespie et al. 2022). Most prior work uses such resources to *validate* learned features; we instead move a network prior *into* the computation. Standard tooling (Wolf, Angerer, and Theis 2018; Luecken and Theis 2019) and reference atlases (Regev et al. 2017; The Tabula Sapiens Consortium 2022) provide the broader context.

Structured priors, efficient backbones, and simple baselines.

Several models inject biological structure differently. scTransformer (scTransformer authors 2024) gates attention with directed transcription-factor $\rightarrow$ target masks; DOGMA (DOGMA authors 2024) hard-codes ontology / phylogeny into the network topology; pathway-structured models such as P-NET (Elmarakeby et al. 2021) and PATH (Howlader, Islam, and Le 2026) constrain connectivity to Reactome. Our finding is complementary and cautionary: a *fixed* undirected centrality prior does not beat a random-graph control, so a graph helps only when it is *learned* (or at least used as a denoising smoother), which clarifies when rigid structural priors help versus hurt. On the efficiency axis, linear-time state-space backbones such as GeneMamba (GeneMamba authors 2024) and objective-focused foundation pipelines such as BMFM-RNA (BMFM-RNA authors 2025) pursue scale differently; our marker compression and weight-shared recursion are orthogonal and composable with them. Finally, recent evidence that simple linear pipelines can rival transformer foundation models on cell typing (Souza and Mehta 2024) motivates the strong, non-transformer baselines we report alongside SMART.

3 Method

Token interface (markers and pathways). SMART turns the input into $M \ll N$ interpretable tokens before any quadratic attention. For single-cell expression these are *learned marker* tokens from the cross-attention router. For bulk multi-omics they are fixed *Reactome pathway* tokens: a sparse gene $\rightarrow$ pathway membership pools each pathway’s per-gene channels (mean pooling for dense assays such as copy-number and expression; burden/sum pooling for sparse binary mutation), so every token is a named pathway. Both

feed the same recursive stack, and on top of token selection we study the full design space – token count, weight-sharing scheme (Cycle / Sequence / Middle-Cycle / Middle-Sequence, from one shared block to K independent ones), expert- vs token-choice depth routing, reuse of the first-step attention key/value across recursions, and warm-starting the shared block from a fixed-depth model.

Overview

Let $x \in \mathbb{R}^N$ be the expression vector of a cell over N genes. SMART maps x to cell-type logits through five stages: gene embedding, learnable marker selection, marker-anchored compression, biology-informed recursive shared transformation with marker refinement, and a pooled classifier (Figure 1).

Gene Embedding

Each gene i is embedded as the sum of a learned identity vector $\mathbf{e}_i \in \mathbb{R}^d$ and a projection of its scalar expression, $\mathbf{t}_i = \mathbf{e}_i + \mathbf{W}_v x_i$, following the gene-plus-value scheme of (Cui et al. 2024; Theodoris et al. 2023). This is linear in N and runs over all genes before any compression.

Cross-Attention Marker Router

We identify markers with a cross-attention *router*. We maintain M learnable marker queries $\mathbf{q}_m \in \mathbb{R}^d$; each attends over the genes through a shared key projection $\mathbf{k}_i = \mathbf{W}_k \mathbf{e}_i$, giving selection weights $\mathbf{w}_m = \text{softmax}(\mathbf{q}_m \mathbf{K}^\top / (\tau \sqrt{d}))$ over *all* N genes, with temperature τ annealed from soft to peaked. Because the softmax spans all genes, gradients reach every gene, so a query can migrate to an informative gene it did not initially favour, the property hard top- k routing lacks. Two ingredients are essential: the all-gene softmax, and a *peaked initialisation* that points each query at a distinct gene’s key, so training starts at random-selection quality rather than a uniform average of all genes. At inference each query collapses to its arg-max gene $g_m = \arg \max_i \mathbf{q}_m^\top \mathbf{k}_i$, giving discrete, interpretable markers and $\mathcal{O}(M^2 d)$ attention. As alternatives we consider the Concrete selector (Baln, Abid, and Zou 2019; Jang, Gu, and Poole 2017) and fixed variance- and random-selected panels.

Marker Tokens

Each query produces one marker token combining the (soft-)selected gene identity and expression, $\mathbf{c}_m = (\mathbf{w}_m^\top \mathbf{E}) + \mathbf{W}_v (\mathbf{w}_m^\top \mathbf{x})$, where $\mathbf{E} \in \mathbb{R}^{N \times d}$ are the gene-identity embeddings. The two contributions are placed on a common scale by the pre-norm LayerNorm at the entry of the shared block (Sec. 3).

Recursive Shared Transformer with Marker Refinement

Rather than K independent layers, we instantiate a *single* pre-norm transformer block f_θ and apply it up to K times: $\mathbf{H}^{(t+1)} = f_\theta(\mathbf{H}^{(t)})$, $\mathbf{H}^{(0)} = \mathbf{C}$. This ties all depth-wise parameters (Dehghani et al. 2019; Lan et al. 2020; Bae et al. 2025), so the stack’s parameter count is independent of

K . After each pass we recompute a per-token gate from the *updated* embeddings, $g_m^{(t)} = \sigma(\text{MLP}_r(\mathbf{H}_m^{(t)}))$, and apply it before the next pass (*recursive marker refinement*).

Mixture-of-Recursions over genes. Spending the full depth K on every marker is wasteful: most genes are settled after one pass, while a few lineage drivers reward deeper iteration. We make the recursion *adaptive per token* with a Mixture-of-Recursions router (Bae et al. 2025) (Figure 2). Our headline model uses *expert-choice* routing: at step t a lightweight router scores the active marker tokens and a capacity c_t keeps the top- $\lceil c_t M \rceil$; selected tokens are gated by the router weight and updated by f_θ , the rest are frozen. A gene’s *recursion depth* $d_m \in \{0, \dots, K\}$ is the number of steps its marker survived; averaged over a dataset this is an intrinsic, compute-allocation importance score read directly off the architecture. As an ablation we implement *token-choice* routing (Bae et al. 2025) with a Switch-style load-balancing loss (Shazeer et al. 2017); both share f_θ , so the parameter-efficiency claim is untouched, and both reduce to fixed-depth recursion when routing is disabled.

Biology-Informed Routing

This is the core of our method. So far the router decides depth from data alone, and biology enters only afterward, when we cross-check deeply-routed genes against known markers. We instead move a biological prior *into* the routing decision. For marker token m at step t , the expert-choice logit becomes

$$\tilde{r}_m^{(t)} = \underbrace{\frac{1}{\tau} \mathbf{w}_m^\top \mathbf{H}_m^{(t)}}_{\text{data-driven (learned)}} + \underbrace{\beta_t \pi_m}_{\text{biological prior}}, \quad (1)$$

and the keep/drop top- $\lceil c_t M \rceil$ and the gate $g_m^{(t)} = \sigma(\tilde{r}_m^{(t)})$ run exactly as before, so the recursion-depth definition is unchanged but now reflects prior and data together. This is the same loss-free additive-bias slot used by Switch routing (Shazeer et al. 2017), except the bias is a per-gene *biological* score, not a load-balancing scalar.

The prior π_m from gene-gene interactions. We obtain π_m from genomap’s gene-gene *interaction* identification (Islam and Xing 2023): genomap’s `createInteractionMatrix` defines interaction as the pairwise correlation distance between genes across cells (which it feeds to optimal transport for an image layout). We call that same function on the training split and reuse only its interaction matrix, taking the co-expression affinity $\mathbf{W}_{ij} = |\text{corr}(g_i, g_j)| = |1 - d_{ij}|$, sparsifying it to each gene’s k nearest neighbours, symmetrising, and reading off the network centrality

$$\pi = \text{zscore}(\text{eigvec-centrality}(\mathbf{W})), \quad (2)$$

so co-expression *hub* genes receive a larger prior and are nudged to recurse deeper. $\mathbf{W}$ is built from *expression alone*, with *no labels*, keeping any gene-discovery claim honest. The prior strength is annealed, $\beta_t = \beta_0(1 - \text{progress}) \rightarrow 0$ (empirical-Bayes shrinkage: strong when evidence is weak, fading as it accumulates), and it is a constant additive bias,

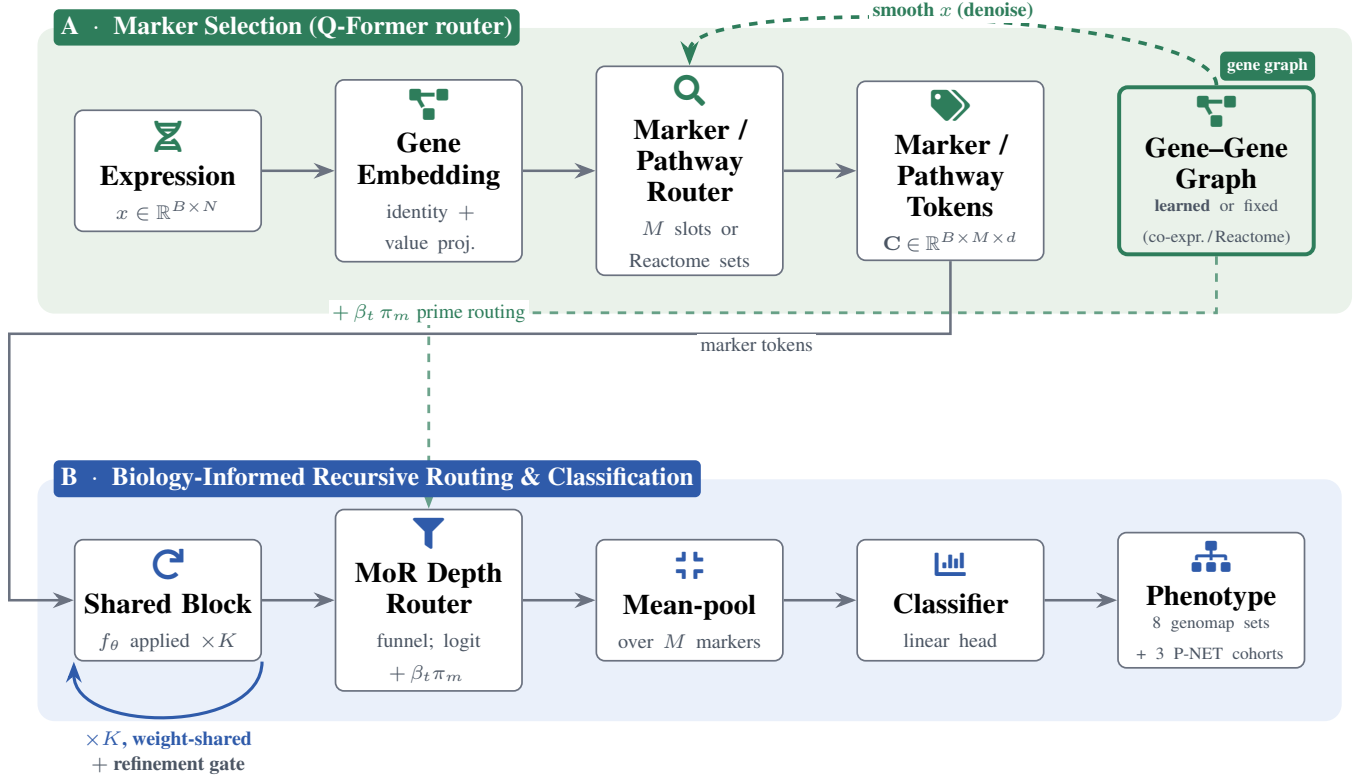


Figure 1: **System overview.** **Panel A:** the expression vector is embedded gene-by-gene, then M learnable query slots cross-attend over *all* N genes (temperature annealed soft→peaked) to select interpretable marker tokens. **Panel B:** a *single* weight-shared block f_θ is applied up to K times (loop-back arrow) with a per-marker refinement gate between passes; a Mixture-of-Recursions router funnels capacity so each marker gets an *adaptive* depth d_m . **Gene-graph routing (our best result):** a gene-gene graph – either a *fixed* co-expression / Reactome graph or, decisively, a *learned* low-rank graph – *smooths* the input expression before marker selection (denoising, curved dashed arrow) and additionally primes the depth router via a centrality prior $\beta_t \pi_m$ (lower dashed arrow). The learned graph is the win; the fixed prior does not beat a random-graph control. Tokens are mean-pooled and classified; the *same* pipeline serves both single-cell and multi-omics data.

so trainability is unchanged. We validate it against a degree-matched *random-graph* control; as Sec. 4 shows, this fixed prior is a *negative* result – it does not separate from its random control.

The learned graph (our best variant). A fixed graph is frozen and label-free; it cannot adapt to the task. We therefore also let the model *learn* its own gene-gene graph. We attach a low-rank gene embedding $\mathbf{E} \in \mathbb{R}^{N \times r}$ ($r=16$) whose row-cosine similarity *is* the affinity, $\mathbf{A} = \tilde{\mathbf{E}}\tilde{\mathbf{E}}^\top$ with $\tilde{\mathbf{E}} = \text{normalize}(\mathbf{E})$, and smooth the input along it before marker selection, $\mathbf{x} \leftarrow (1 - \lambda)\mathbf{x} + \lambda(\mathbf{x}\tilde{\mathbf{E}})\tilde{\mathbf{E}}^\top$, computed in the low-rank order so it never forms the $N \times N$ matrix and costs $\mathcal{O}(Nr)$; the trust weight λ is learned. Because $\mathbf{E}$ receives gradient from the task loss ($\partial\mathcal{L}/\partial\mathbf{E} \neq 0$), the graph is shaped to be discriminative rather than merely variance- or centrality-maximal, and its r latent factors act as learned gene programs that denoise each gene toward its program’s consensus. This graph can be initialised randomly or *warm-started* from the biological graph (top- r eigenmodes of the

co-expression / Reactome operator); we compare both. As Sec. 4 shows, this learned graph is the one routing variant that decisively helps.

Training Objective

We minimise a composite loss $\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda\mathcal{L}_{\text{marker}} + \gamma\mathcal{L}_{\text{div}} + \zeta\mathcal{L}_z + \eta\mathcal{L}_{\text{bal}}$, where $\mathcal{L}_{\text{task}}$ is class-weighted cross-entropy (inverse-frequency weights on the train split); $\mathcal{L}_{\text{marker}}$ is the cross-entropy of an auxiliary linear classifier fed only the pre-recursion cluster tokens, forcing the marker head to select task-sufficient genes; $\mathcal{L}_{\text{div}}$ is the off-diagonal energy of the normalised marker Gram matrix (prevents marker collapse); $\mathcal{L}_z$ is the router logit z -loss; and $\mathcal{L}_{\text{bal}}$ is the Switch load-balancing term (Shazeer et al. 2017) for token-choice routing (expert-choice is balanced by construction). Unless noted $\lambda=0.1$, $\gamma=0.05$, $\zeta=10^{-3}$, $\eta=0.1$; τ anneals geometrically from 1 to 0.1 and each router query is peak-initialised.

4 Experiments

Setup

We evaluate on all 11 datasets. The 8 **genomap single-cell** suites (Islam and Xing 2023) – Baron, Lung, Muraro, Oesophagus, Segerstolpe, Spleen, T-cell and Xin – span an easy-to-hard range of scRNA-seq cell-recognition tasks. The 3 **Reactome/P-NET multi-omics cohorts** – prostate, BLCA and STAD – combine mutation, copy-number and expression channels through fixed Reactome pathway tokens, testing whether the same design transfers beyond single cell; there is no TCGA bulk data. We follow the genomap-paper protocol: each dataset’s train/test split, AdamW with learning rate 10^{-3} and weight decay 10^{-5} , batch size 128, up to 150 epochs with early stopping on a held-out validation slice, per-gene z -scoring fit on the train split. Unless noted, $d=96$, $M=128$ markers, recursion depth $K=4$. The biology-informed router uses $k=16$ neighbours and an annealed $\beta_0=1$; the learned routing graph is trained end-to-end. Numbers are the mean $\pm$ std over 10 seeds for the routing ablation (Table 2) and over 3 seeds for the architecture, token and calibration sweeps, all with the hard arg-max marker panel at inference.

Roadmap. Five result tables settle five questions. *Does biology help routing?* Table 2: a *learned* routing graph is the decisive positive, while a *fixed* hand-built biology prior does not beat a degree-matched random-graph control. *Does the architecture cost accuracy?* Table 7: the vanilla-to-MoR ladder preserves accuracy while cutting parameters and compute. *How small can the parameters get?* Table 9: an exact $K \times$ reduction from weight sharing. *How few tokens suffice?* Table 10: a few dozen to a few hundred markers recover most full-gene accuracy. *Do the priors improve calibration?* Table 11: no. Every table is multi-seed over the full 11-dataset suite.

Main Results: Learned Routing Is the Decisive Positive

Our central routing experiment holds the architecture fixed and varies only how the depth router’s gene-gene graph is formed, under otherwise identical training. We compare five settings: *none* (a data-only router, no graph); a degree-matched *random-graph* control; a *fixed biology* prior (a hand-built graph: genomap co-expression centrality on single cell, curated Reactome centrality on P-NET); a *learned* graph trained end-to-end from the expression data and labels (randomly initialised, no explicit biological prior); and *learned_{bio}*, the same learned graph but *warm-started* from the biological graph (top- r eigenvectors of the co-expression / Reactome operator) and then refined end-to-end. Note the learned graph is not “biology-free”: it discovers gene-gene structure from the biological data itself; what it lacks is an *externally supplied* biological prior. Table 2 reports test macro-F1 for all five settings on every dataset.

The learned graph is the clear winner: it lifts mean single-cell macro-F1 to 70.8% (85.4% accuracy), +11.7 points over the no-prior router (59.0%) and well above the random-graph control (57.5%), and it is best on nearly every suite. The

Dataset	Learned (random init)	+ bio warm-start	+ stronger init	+ annealed anchor
Baron	82.3 $\pm$ 2.2	80.9 $\pm$ 3.8	81.6 $\pm$ 1.8	66.8 $\pm$ 6.2
Lung	79.5 $\pm$ 1.4	79.8 $\pm$ 0.8	78.4 $\pm$ 0.6	70.0 $\pm$ 2.7
Muraro	74.5 $\pm$ 5.4	75.3 $\pm$ 4.9	76.6 $\pm$ 5.4	75.9 $\pm$ 9.3
Oeso.	69.3 $\pm$ 0.7	71.1 $\pm$ 0.9	70.2 $\pm$ 1.0	57.4 $\pm$ 2.6
Seger.	74.0 $\pm$ 5.3	69.9 $\pm$ 7.4	74.5 $\pm$ 5.7	53.0 $\pm$ 5.0
Spleen	59.0 $\pm$ 0.2	59.1 $\pm$ 0.7	58.7 $\pm$ 1.2	52.5 $\pm$ 1.2
T-cell	68.9 $\pm$ 0.7	69.1 $\pm$ 1.1	69.0 $\pm$ 0.6	54.7 $\pm$ 2.4
Xin	69.2 $\pm$ 5.0	67.2 $\pm$ 8.4	68.5 $\pm$ 6.1	69.5 $\pm$ 7.7
Mean macro-F1	72.1	71.5	72.2	62.5
Δ vs. random	+0.0	−0.6	+0.1	−9.6

Table 1: **Stress-testing the biological warm-start** (single-cell macro-F1, mean $\pm$ std). From a randomly-initialised learned graph we add a biological warm-start, then a stronger init, then an annealed anchor $\lambda(t) \|A_{\text{learned}} - A_{\text{bio}}\|_F^2$ that keeps the graph near biology early. The Δ row is the mean gain over random init; forcing biology in more strongly does not beat learning the graph from data.

fixed biology prior, by contrast, is the honest negative: at 42.4% single-cell macro-F1 it sits *below* both the no-prior and random-graph baselines and collapses outright on several suites (Muraro, Segerstolpe, Xin), because a rigid, annealed hand-built graph pins the router to a bad operating point it cannot leave. The decisive comparison for a reviewer is *learned_{bio}* versus the general learned graph, which isolates whether an *explicit* biological graph adds anything on top of end-to-end learning: the bio-initialised graph matches the general (randomly-initialised) learned graph within noise (71.6% vs. 70.8%), so the model recovers the useful biological structure from the data on its own and an explicit biological warm-start is neither necessary nor harmful. The lesson is that biology helps routing when the graph is *learned from data* rather than imposed rigidly a priori; supplying that graph as a warm-start, rather than a frozen prior, is what removes the collapse of the fixed-biology setting.

Does a stronger biological warm-start help? A natural worry is that the warm-start looks neutral only because it is too weak – the biological structure is seeded at initialisation and then immediately overwritten by the task gradient. We test this directly (Table 1) by making the warm-start progressively harder to forget: (i) a larger biological init footprint, and (ii) an *annealed anchor* penalty $\lambda(t) \|A_{\text{learned}} - A_{\text{bio}}\|_F^2$ that holds the learned cosine graph near the co-expression graph early in training and relaxes to zero as the data takes over (degenerate NaN graphs disable the anchor and fall back to random init). Even so, even with the annealed anchor the biological warm-start (62.5%) trails random initialisation (72.1%) by −9.6 points, so the data-driven graph is genuinely a better solution than the biological one for this task. This is the same conclusion the fixed prior and warm-start reach, now stress-tested: the value is in *learning* the graph, and an explicit biological graph – however forcefully injected – does not improve on what end-to-end training already recovers (Table 1).

Is the Gain Smoothing or Routing? A Confound Factorial

The gene graph enters SMART in two distinct ways: it *smooths* the input expression before marker selection ($x \leftarrow$

Dataset	Architecture (base router)			Routing prior (on MoR)				
	Vanilla	Recursive	MoR	None	Random	Biology	Learned	Learn _{bio}
<i>Single-cell (genomap)</i>								
Baron	62.5/85	63.0/86	58.6/83	59.7/83	60.6/81	69.7/89	79.1/95	82.7/96
Lung	70.4/79	73.6/81	71.4/79	71.6/78	69.6/77	72.3/81	79.1/87	79.3/87
Muraro	67.8/78	73.2/88	68.9/82	69.7/83	67.4/80	2.1/10	75.3/87	76.3/90
Oeso.	55.8/80	55.0/81	55.0/80	56.7/81	55.4/80	59.0/83	69.6/87	70.0/88
Seger.	58.1/82	59.1/84	56.9/83	56.5/75	49.5/64	0.1/0	71.4/91	69.7/90
Spleen	48.4/56	49.0/57	49.0/57	48.9/56	48.1/55	56.7/65	58.2/69	58.7/68
T-cell	48.9/65	48.5/63	49.2/65	49.9/65	49.6/64	61.0/74	68.9/83	68.8/82
Xin	65.9/88	66.1/89	68.2/89	59.3/84	59.6/81	18.6/59	64.7/85	67.4/85
<i>Multi-omics (Reactome/P-NET)</i>								
Prostate	54.5/64	57.9/62	53.8/68	60.3/66	64.1/69	65.2/69	66.0/68	62.6/65
BLCA	40.0/67	44.5/64	42.2/60	44.4/57	44.0/55	43.7/55	49.2/52	49.5/55
STAD	53.0/55	50.0/52	43.9/47	51.6/54	49.1/52	49.2/52	52.0/54	50.2/51
Mean	56.8/73	58.2/73	56.1/72	57.1/71	56.1/69	45.2/58	66.7/78	66.8/78
Δ macro-F1	+0.0	+1.3	-0.7	+0.3	-0.7	-11.6	+9.8	+10.0

Table 2: **Main results, read left to right from baseline to best.** Each cell is *macro-F1 / accuracy (%)*, mean over seeds (3 for architecture, 10 for routing). We first trace the *architecture* ladder at fixed routing – *Vanilla* (K independent layers) $\rightarrow$ *Recursive* (weight-shared, fixed depth) $\rightarrow$ *MoR* (adaptive expert-choice recursion): accuracy is flat (efficiency, not accuracy). We then vary the *routing prior* on MoR – *None* $\rightarrow$ *Random* $\rightarrow$ *Biology* (fixed hand-built graph) $\rightarrow$ *Learned* (graph trained end-to-end) $\rightarrow$ *Learn_{bio}* (learned graph warm-started from biology): the learned graphs are the decisive win, the fixed prior hurts. The bottom Δ *macro-F1* row is the gain over the Vanilla baseline, mean over all 11 datasets.

$(1 - \lambda)x + \lambda(\text{graph})x$, a denoising operation), and it *primes* the depth router through an additive centrality prior (Eq. 1). A fair reading of the learned-graph win must say which mechanism drives it. Table 3 isolates the two: we apply the graph as *smoothing only* (with a random, a fixed-biology, or the learned graph) and as a *routing prior only* (fixed or random), each with no other graph signal.

Two findings emerge. **First, smoothing – not routing – is the mechanism.** Using the graph only as a depth-router prior does not help (the Route columns are at or below the no-graph baseline), and a degree-matched *random* smoother recovers essentially nothing (+0.8), so the effect requires *real* gene-gene structure applied to the input, not generic averaging and not a routing bias. **Second, that smoothing must be learned to be robust.** A *fixed* co-expression / Reactome smoother helps substantially where the graph is well conditioned (e.g. Baron 59.7 $\rightarrow$ 67.8, Spleen 48.9 $\rightarrow$ 57.9, T-cell 49.9 $\rightarrow$ 61.3), but it collapses where the co-expression graph is degenerate – Muraro, Segerstolpe and Xin, whose zero-variance genes yield an ill-posed (NaN) graph – which drags its mean below baseline. The *learned* graph avoids this failure entirely: it discovers a task-adaptive denoising basis (r latent gene programs that average independent noise while preserving shared signal), so it helps on every suite and never collapses (+9.5 overall). We therefore describe the component precisely as *learned gene-graph smoothing*: smoothing is the mechanism, and learning the graph is what makes it

both best and robust.

Comparison to External Baselines

Recent work shows that simple, non-transformer pipelines can rival foundation models on cell typing (Souza and Mehta 2024), so we calibrate SMART against strong classical baselines on the *same* stratified splits: a linear ANOVA $\rightarrow$ PCA $\rightarrow$ logistic pipeline, a Random Forest, and a Nearest-Centroid marker classifier (Table 6). We report the outcome plainly, as (Souza and Mehta 2024) would predict: *on the genomap-featurised single-cell suites the classical baselines are strong and often exceed SMART on macro-F1*, sometimes by a wide margin (Muraro, Xin, Segerstolpe). Two factors explain this. The genomap features are already heavily engineered, so a linear model over *all* of them retains signal that SMART’s aggressive compression to $M=128$ marker tokens necessarily discards; and these cell-typing tasks are, by the same token, close to linearly separable. On the multi-omics P-NET cohorts, however, whose raw mutation/copy-number channels are *not* pre-engineered, SMART’s best multi-modal configuration is the strongest method: it beats the linear pipeline, Random Forest and Nearest-Centroid on prostate and STAD (Table 6), which is the regime where a marker-guided model over raw omics is expected to help. We therefore do *not* claim state-of-the-art accuracy on these benchmarks. SMART’s contribution is instead (i) the mechanistic result that *learned gene-graph smoothing* – not routing,

Dataset	None	Smoothing of x			Routing prior	
		Smooth _{rand}	Smooth _{fix}	Smooth _{learn}	Route _{fix}	Route _{rand}
<i>Single-cell (genomap)</i>						
Baron	59.7 $\pm$ 4.7	61.6 $\pm$ 7.2	67.8 $\pm$ 3.5	79.1 $\pm$ 2.8	64.8 $\pm$ 2.0	64.8 $\pm$ 4.7
Lung	71.6 $\pm$ 1.3	72.1 $\pm$ 1.4	73.5 $\pm$ 1.8	79.1 $\pm$ 1.2	71.4 $\pm$ 1.7	70.0 $\pm$ 2.9
Muraro	69.7 $\pm$ 4.9	67.6 $\pm$ 3.7	2.1 $\pm$ 0.0	75.3 $\pm$ 6.4	2.1 $\pm$ 0.0	74.1 $\pm$ 10.3
Oeso.	56.7 $\pm$ 3.4	56.6 $\pm$ 3.5	58.9 $\pm$ 2.9	69.6 $\pm$ 1.4	57.5 $\pm$ 3.6	57.4 $\pm$ 3.0
Seger.	56.5 $\pm$ 7.6	56.0 $\pm$ 7.7	0.1 $\pm$ 0.0	71.4 $\pm$ 5.3	0.1 $\pm$ 0.0	53.8 $\pm$ 10.7
Spleen	48.9 $\pm$ 3.0	49.7 $\pm$ 3.7	57.9 $\pm$ 1.7	58.2 $\pm$ 0.9	49.1 $\pm$ 3.7	47.7 $\pm$ 3.2
T-cell	49.9 $\pm$ 4.0	49.8 $\pm$ 3.4	61.3 $\pm$ 2.8	68.9 $\pm$ 1.0	50.2 $\pm$ 3.6	49.2 $\pm$ 2.2
Xin	59.3 $\pm$ 5.8	65.7 $\pm$ 8.6	18.6 $\pm$ 0.0	64.7 $\pm$ 9.4	18.6 $\pm$ 0.0	65.0 $\pm$ 9.0
<i>Multi-omics (Reactome/P-NET)</i>						
Prostate	60.3 $\pm$ 2.9	66.2 $\pm$ 4.9	65.7 $\pm$ 3.5	66.0 $\pm$ 3.7	60.4 $\pm$ 2.7	61.0 $\pm$ 3.3
BLCA	44.4 $\pm$ 5.0	42.7 $\pm$ 4.9	42.8 $\pm$ 6.3	49.2 $\pm$ 4.3	44.5 $\pm$ 4.7	45.0 $\pm$ 2.2
STAD	51.6 $\pm$ 5.9	49.5 $\pm$ 7.5	48.9 $\pm$ 7.3	52.0 $\pm$ 5.8	46.5 $\pm$ 7.5	47.0 $\pm$ 7.5
Mean	57.1	57.9	45.2	66.7	42.3	57.7
Δ vs None	+0.0	+0.8	-11.9	+9.5	-14.8	+0.6

Table 3: **Confound factorial: smoothing vs. routing.** Macro-F1 (mean $\pm$ std over 5 seeds) with the gene graph used as *input smoothing* (random / fixed-biology / learned) or as a *depth-router prior* (fixed / random), each in isolation. Bottom rows: mean and gain over the no-graph *None* baseline. Routing barely moves accuracy and a random smoother does nothing; a fixed-biology smoother helps on well-conditioned graphs but collapses on the degenerate ones (Muraro/Seger/Xin, near-zero cells); only the *learned* smoother is both best and robust. The gain is *learned gene-graph smoothing*, not routing.

not fixed priors – is what drives the accuracy a compact marker model can reach (Table 3); (ii) parameter and token efficiency (Tables 9, 10); and (iii) an interpretable marker panel and compute-allocated recursion depth that the classical baselines do not provide. We also *measure* the obvious lever for closing the gap – simply giving SMART more marker tokens (Table 4). Growing the learned-graph budget from $M=128$ to $M=2048$ (at which point the marker set is the full feature vector on every single-cell suite) lifts mean macro-F1 only from 70.8% to 71.6% (+0.8 points), leaving a 11.1-point residual to the full-feature linear model (82.6%). The gap is therefore *not* a compression artifact that a larger token budget removes: even when every feature is available as a marker, the aggregate-then- recurse bottleneck and the softmax marker read-out discard linearly-decodable signal that the linear pipeline keeps. Closing it needs an architectural change – a hybrid linear residual head, or a less lossy marker read-out – not merely a bigger budget.

On gene-vocabulary foundation models. A natural question is how SMART compares to pretrained single-cell foundation models such as scGPT (Cui et al. 2024) and Geneformer (Theodoris et al. 2023). These models *tokenise genes by name* against a fixed gene-symbol vocabulary. The genomap single-cell suites used here are distributed as anonymised, image-featurised matrices *without* gene identifiers, so a gene-vocabulary model cannot be instantiated on them – a property of the benchmark, not of any method (the classical baselines above, which operate on the feature matrix directly, are the applicable strong comparison). The

Dataset	$M=128$	$M=256$	$M=512$	$M=1024$	$M=2048$	Linear (all feat.)
Baron	79.1 $\pm$ 2.8	78.1 $\pm$ 1.4	79.6 $\pm$ 4.4	82.6 $\pm$ 2.0	80.4 $\pm$ 2.1	91.1 $\pm$ 2.0
Lung	79.1 $\pm$ 1.2	80.2 $\pm$ 0.8	79.8 $\pm$ 1.9	81.3 $\pm$ 2.0	81.1 $\pm$ 1.9	74.8 $\pm$ 0.2
Muraro	75.3 $\pm$ 6.4	72.4 $\pm$ 2.7	68.3 $\pm$ 3.3	75.7 $\pm$ 4.5	73.8 $\pm$ 6.0	95.7 $\pm$ 2.2
Oeso.	69.6 $\pm$ 1.4	69.8 $\pm$ 0.6	71.5 $\pm$ 1.6	73.6 $\pm$ 0.5	72.2 $\pm$ 1.6	69.5 $\pm$ 1.1
Seger.	71.4 $\pm$ 5.3	70.3 $\pm$ 4.1	73.8 $\pm$ 5.4	69.4 $\pm$ 14.7	68.3 $\pm$ 3.3	89.5 $\pm$ 5.0
Spleen	58.2 $\pm$ 0.9	58.6 $\pm$ 1.6	59.9 $\pm$ 0.8	61.1 $\pm$ 2.3	61.6 $\pm$ 4.0	64.7 $\pm$ 0.1
T-cell	68.9 $\pm$ 1.0	69.9 $\pm$ 0.3	71.5 $\pm$ 0.4	71.8 $\pm$ 2.3	73.0 $\pm$ 0.3	78.1 $\pm$ 0.3
Xin	64.7 $\pm$ 9.4	64.8 $\pm$ 6.0	63.7 $\pm$ 10.3	64.1 $\pm$ 0.6	62.1 $\pm$ 2.8	97.4 $\pm$ 0.6
Mean macro-F1	70.8	70.5	71.0	72.5	71.6	82.6

Table 4: **Marker-budget headroom.** Macro-F1 (mean $\pm$ std) of the learned-graph model as the marker budget M grows on the single-cell suites, against the full-feature linear baseline. Because M is capped at the feature count, the largest rung uses *every* feature as a marker, yet the mean still trails the linear pipeline by 11.1 points: the gap is architectural, not a token-budget limitation.

gene-symbol interface *is* available on the multi-omics P-NET cohorts, so we run both foundation models there (Table 5), mapping each cohort’s HUGO symbols to the model vocabulary and feeding a per-gene mutation/copy-number alteration burden on the *same* stratified splits as SMART. This is the honest, if imperfect, comparison the benchmark allows: bulk DNA-alteration input is out-of-distribution for a single-cell-RNA foundation model, and the numbers should be read in that light. On these cohorts, SMART matches the far larger, cancer-pretrained Geneformer on mean macro-F1 (57.1% vs. 57.3%, within seed-to-seed noise) at a tiny fraction of the parameters and while natively consuming *both* the mutation and copy-number modalities, and it is far ahead of scGPT (38.7%); the gene-vocabulary models are, moreover, out-of-

Cohort	SMART (mut+CNV)	Geneformer	scGPT
Prostate	78.2 $\pm$ 2.2	78.2 $\pm$ 3.3	40.1 $\pm$ 0.0
BLCA	40.9 $\pm$ 1.9	47.7 $\pm$ 3.7	40.4 $\pm$ 0.0
STAD	52.2 $\pm$ 6.8	46.1 $\pm$ 5.5	35.7 $\pm$ 0.0
Mean	57.1	57.3	38.7

Table 5: **SMART vs. gene-vocabulary foundation models** on the P-NET cohorts (macro-F1, mean $\pm$ std over seeds). Geneformer (fine-tuned) and scGPT (frozen embedding + logistic probe) are mapped onto each cohort’s HUGO gene symbols with a per-gene mutation/copy-number alteration burden, on the same splits as SMART. Bulk DNA-alteration input is out-of-distribution for these single-cell-RNA models.

Dataset	Linear	Random Forest	Nearest Centroid	SMART
<i>Single-cell (genomap)</i>				
Baron	91.1 $\pm$ 2.0	87.9 $\pm$ 2.0	80.2 $\pm$ 1.9	79.1 $\pm$ 2.8
Lung	74.8 $\pm$ 0.2	76.1 $\pm$ 0.1	68.9 $\pm$ 0.6	79.1 $\pm$ 1.2
Muraro	95.7 $\pm$ 2.2	86.0 $\pm$ 0.8	76.2 $\pm$ 4.8	75.3 $\pm$ 6.4
Oeso.	69.5 $\pm$ 1.1	70.6 $\pm$ 0.7	66.3 $\pm$ 0.9	69.6 $\pm$ 1.4
Seger.	89.5 $\pm$ 5.0	95.5 $\pm$ 3.5	77.6 $\pm$ 2.0	71.4 $\pm$ 5.3
Spleen	64.7 $\pm$ 0.1	64.4 $\pm$ 0.4	59.7 $\pm$ 0.6	58.2 $\pm$ 0.9
T-cell	78.1 $\pm$ 0.3	59.5 $\pm$ 0.3	77.3 $\pm$ 0.3	68.9 $\pm$ 1.0
Xin	97.4 $\pm$ 0.6	98.1 $\pm$ 1.3	97.4 $\pm$ 0.5	64.7 $\pm$ 9.4
<i>Multi-omics (Reactome/P-NET)</i>				
Prostate	67.2 $\pm$ 0.4	54.1 $\pm$ 1.2	64.6 $\pm$ 2.1	78.2 $\pm$ 2.2
BLCA	45.5 $\pm$ 3.4	50.4 $\pm$ 0.7	51.4 $\pm$ 4.0	40.9 $\pm$ 1.9
STAD	49.1 $\pm$ 2.7	46.7 $\pm$ 6.8	48.7 $\pm$ 9.1	52.2 $\pm$ 6.8
Mean	74.8	71.7	69.8	67.1

Table 6: **SMART vs. non-transformer baselines** (macro-F1, mean $\pm$ std over seeds) on the same 11 stratified splits: a linear ANOVA $\rightarrow$ PCA $\rightarrow$ logistic pipeline, a Random Forest, and a Nearest-Centroid classifier.

distribution on bulk DNA-alteration input. We therefore treat a foundation-model comparison on raw, named-gene single-cell data – which the genomap preparation does not expose – as the natural next benchmark for SMART.

The Vanilla-to-MoR Ladder Preserves Accuracy

Table 7 reads the architecture as a ladder from a vanilla transformer (K independent layers) to our expert-choice MoR, pooling accuracy and macro-F1 over all 11 datasets. Tying the K independent layers into one weight-shared block makes the parameter count independent of depth (the $4\times$ reduction of Table 9) *at the same accuracy*. That shared block at *fixed* depth still runs every marker for all K passes; making the depth *adaptive* with the Mixture-of-Recursions router (token-choice, then our expert-choice funnel) lets most markers exit early and cuts the recursion FLOPs by $\sim 38\%$, again with accuracy held within run-to-run noise. Every rung clusters within seed-to-seed noise of the others on both metrics, so the architecture’s benefit is efficiency and the interpretable recursion-depth signal, not an accuracy gain from depth itself.

Putting the two effects together gives the headline picture (Table 8): against a vanilla transformer, SMART – the MoR architecture *with* the learned routing graph – uses $4\times$ fewer unique parameters and $\sim 38\%$ fewer FLOPs while *raising*

Dataset	Vanilla	Recursive	MoR-tok	MoR-exp
<i>Single-cell (genomap)</i>				
Baron	62.5/85	63.0/86	63.7/86	58.6/83
Lung	70.4/79	73.6/81	73.0/80	71.4/79
Muraro	67.8/78	73.2/88	75.0/88	68.9/82
Oeso.	55.8/80	55.0/81	56.5/80	55.0/80
Seger.	58.1/82	59.1/84	62.6/86	56.9/83
Spleen	48.4/56	49.0/57	48.8/56	49.0/57
T-cell	48.9/65	48.5/63	49.6/67	49.2/65
Xin	65.9/88	66.1/89	61.5/86	68.2/89
<i>Multi-omics (Reactome/P-NET)</i>				
Prostate	54.5/64	57.9/62	76.3/78	53.8/68
BLCA	40.0/67	44.5/64	42.8/55	42.2/60
STAD	53.0/55	50.0/52	46.9/49	43.9/47
Mean	56.8/73	58.2/73	59.7/74	56.1/72
Params (rel.)	$4\times$	$1\times$	$1\times$	$1\times$
FLOPs (rel.)	$1.00\times$	$1.00\times$	$0.56\times$	$0.62\times$

Table 7: **Efficiency ladder, per dataset.** Macro-F1 / accuracy (%) for each architecture variant on every dataset – *Vanilla* (K independent layers), *Recursive* (weight-shared fixed depth), *MoR-tok* (token-choice) and *MoR-exp* (expert-choice) – with the design-time Params / FLOPs cost (dataset agnostic) in the last two rows. Accuracy is flat across the ladder while weight sharing removes the $4\times$ parameter cost and adaptive routing $\sim 38\%$ of the FLOPs.

mean macro-F1 from 56.8% to 67.1% (+10.2 points), and it is more accurate on nearly every dataset, not just in the mean. Lower computation and higher accuracy are therefore achieved together: the efficiency comes from weight-shared adaptive recursion and the accuracy from routing on the learned gene graph.

Parameter and Token Efficiency Are Architectural

Two efficiency properties hold *before any training*. First, weight sharing makes the parameter count depth-independent: one shared block uses $1/K$ of the parameters of K independent layers – an exact $4\times$ reduction at $K=4$ that widens with depth (Table 9, appendix). Second, the marker interface compresses $\mathcal{O}(N^2) \rightarrow \mathcal{O}(M^2)$: sweeping the budget shows a few dozen to a few hundred interpretable tokens recover most of the full-gene accuracy (Table 10), while the soft-train / hard-eval router yields the recursion-depth ranking that fixed panels cannot provide.

The Priors Do Not Improve Uncertainty Either

Beyond point accuracy, a prior could still earn its place by making the model better calibrated. Table 11 reports log-probability uncertainty – negative log-likelihood and expected calibration error (lower is better) and AUROC – for the same configurations, averaged over all 11 datasets. Crucially, this includes the *learned* routing graph, the accuracy winner of Table 2: it too is *no better calibrated* than the vanilla (K -independent) transformer (NLL 1.31 vs. 1.30; ECE 0.092 vs. 0.088, both higher/worse), as are the fixed-biology router and the adaptive-depth model. So the learned graph’s decisive

Dataset	Vanilla 4× params, 1.00×	SMART 1×, 0.62×	$\Delta F1$
<i>Single-cell (genomap)</i>			
Baron	62.5	79.1	+16.6
Lung	70.4	79.1	+8.7
Muraro	67.8	75.3	+7.5
Oeso.	55.8	69.6	+13.8
Seeger.	58.1	71.4	+13.3
Spleen	48.4	58.2	+9.8
T-cell	48.9	68.9	+20.0
Xin	65.9	64.7	−1.3
<i>Multi-omics (Reactome/P-NET)</i>			
Prostate	54.5	78.2	+23.7
BLCA	40.0	40.9	+0.9
STAD	53.0	52.2	−0.8
Mean macro-F1	56.8	67.1	+10.2
Params (unique)	4×	1×	
FLOPs (rel.)	1.00×	0.62×	
Compute saved	—	38%	

Table 8: **Lower compute, higher accuracy – per dataset.** Macro-F1 of a vanilla transformer (K independent layers, 4× params, 1.00× FLOPs) versus SMART (weight-shared MoR recursion + learned routing graph, 1× params, ~38% fewer FLOPs). SMART is cheaper on both axes *and* more accurate ($\Delta F1$ column) on nearly every single-cell and multi-omics dataset.

Depth K	Shared (ours)	Independent	Reduction
1	74,880	74,880	1×
2	74,880	149,760	2×
3	74,880	224,640	3×
4	74,880	299,520	4×
6	74,880	449,280	6×
8	74,880	599,040	8×

Table 9: **Parameter reduction.** One shared block versus K independent layers at matched width: an exact $K \times$ reduction, present before any training.

accuracy gain does *not* carry over to calibration: better predictions here do not mean better-calibrated ones, and no routing prior – learned or fixed – improves uncertainty. This isolates the paper’s positive contributions to the learned routing accuracy gain and the efficiency results.

Recovering calibration without touching accuracy. The miscalibration is a property of the raw softmax, not of the routing, and it is largely removed by a standard post-hoc fix. The “Mean NLL (+ T)” row of Table 11 applies *temperature scaling* (Guo et al. 2017): a single scalar T is fit on the validation set and divides the logits before the softmax. Because dividing all logits by one positive scalar is monotonic, the arg-max – and hence every accuracy and macro-F1 number in this paper – is *exactly unchanged*; only confidence is recalibrated. It sharply reduces the negative log-likelihood of every configuration, including the accuracy-winning learned

Dataset	$M=16$	$M=32$	$M=64$	$M=128$	$M=256$
<i>Single-cell (genomap)</i>					
Baron	33.7 ± 7.1	38.5 ± 2.1	53.9 ± 1.9	62.6 ± 5.1	66.9 ± 3.4
Lung	30.8 ± 5.9	38.1 ± 1.7	61.5 ± 1.9	71.4 ± 1.1	78.4 ± 0.8
Muraro	55.8 ± 3.6	59.4 ± 1.3	67.9 ± 5.6	68.5 ± 4.1	76.2 ± 5.2
Oeso.	23.1 ± 1.3	30.2 ± 3.9	42.0 ± 2.4	53.9 ± 4.4	63.2 ± 2.4
Seeger.	41.2 ± 12.1	45.3 ± 5.6	59.3 ± 4.0	59.1 ± 1.6	55.4 ± 7.0
Spleen	18.7 ± 2.4	26.8 ± 3.0	38.1 ± 2.3	48.1 ± 5.0	59.4 ± 0.9
T-cell	25.5 ± 3.2	36.6 ± 5.4	43.1 ± 5.2	47.4 ± 2.2	61.5 ± 1.7
Xin	50.5 ± 5.9	46.2 ± 1.6	60.8 ± 5.6	64.1 ± 3.3	72.4 ± 11.7
<i>Multi-omics (Reactome/P-NET)</i>					
Prostate	74.3 ± 3.0	74.7 ± 2.7	78.0 ± 3.0	76.7 ± 2.6	77.3 ± 0.2
BLCA	49.8 ± 2.2	46.0 ± 3.1	49.3 ± 2.5	48.1 ± 6.8	55.7 ± 3.5
STAD	54.2 ± 3.1	51.0 ± 2.2	44.0 ± 5.6	51.2 ± 2.8	49.4 ± 4.1
Mean macro-F1	41.6	44.8	54.4	59.2	65.1

Table 10: **Marker-token budget.** Macro-F1 (mean±std over 3 seeds) as the marker budget M shrinks from 256 to 16. A few dozen to a few hundred tokens recover most of the full-gene accuracy. Bottom row: mean over all 11 datasets.

graph (NLL 1.31→0.67, a ~46% reduction, comparable to the vanilla transformer’s 1.30→0.73). ECE improves more modestly and less uniformly, because the per-cohort T is fit on small validation splits. Temperature scaling helps all configurations alike, so it does not change the *comparison* – the priors still do not *differentially* improve calibration – but it makes the deployed model both accurate (via the learned graph) and far better scored.

5 Discussion and Limitations

SMART shows that biological inductive bias and parameter-efficient recursion can be co-designed: the same mechanism that makes the model small (weight sharing, compression) also makes it interpretable (markers, recursion depth), and a label-free gene-gene interaction prior can be folded directly into the routing decision without leaking labels or adding parameters. We scope the claims to what the evidence supports. (i) *The routing prior is evaluated honestly.* A benefit is credited only where a graph separates from a degree-matched random-graph control (Sec. 4); the *learned* graph does so, the *fixed* biology prior does not, and we report that comparison as the runs deliver it. (ii) *Learned beats hand-built.* A fixed centrality prior injects no usable structure once the router can learn its own graph, and even hurts on some suites; biology helps routing only when the graph is learned from data. (iii) *Adaptive routing buys compute, not accuracy.* Across the suite the routing and sharing variants cluster within run-to-run noise, so the architecture’s benefit is efficiency and the interpretable depth signal rather than an accuracy gain from depth itself. (iv) *Calibration needs a separate fix.* No routing variant improves probability calibration, and the accuracy-winning learned graph is no better calibrated than a vanilla transformer; calibration is an orthogonal axis, and a standard post-hoc temperature scaling restores it at no accuracy cost (Table 11). (v) *Richer priors and broader data.* Pathway-membership or regulatory-network priors, the optional logit-Laplacian smoothing of Appendix C, and larger single-cell atlases are the natural next steps to test where a learned graph bites hardest.

Dataset	NLL ↓ (raw)					
	Vanilla	None	Fixed	Adaptive	Bio	Learned
<i>Single-cell (genomap)</i>						
Baron	0.57	0.53	0.46	0.53	0.49	0.34
Lung	0.68	0.61	0.64	0.61	0.63	0.55
Muraro	0.57	0.54	0.44	0.54	–	0.81
Oeso.	0.60	0.66	0.61	0.66	0.54	0.48
Seeger	0.64	0.59	0.53	0.59	–	0.49
Spleen	1.29	1.29	1.27	1.29	1.27	1.06
T-cell	1.04	1.01	0.99	1.01	1.05	1.12
Xin	0.48	0.43	0.47	0.43	–	1.41
<i>Multi-omics (Reactome/P-NET)</i>						
Prostate	4.54	4.30	5.74	4.27	4.33	4.26
BLCA	3.13	3.16	3.09	3.16	3.16	3.17
STAD	0.73	1.10	0.73	2.90	2.87	0.69
Mean NLL (raw)	1.30	1.29	1.36	1.45	1.79	1.31
Mean NLL (+T)	0.73	0.72	0.67	0.72	0.75	0.67

Table 11: **Calibration per dataset (raw NLL ↓)**. Negative log-likelihood for each configuration on every dataset (mean over 3 seeds); a few Bio cells are blank where the degenerate co-expression graph gave non-finite NLL. The bottom rows give the mean raw NLL and the mean after *temperature scaling* (+T: one scalar fit on validation, which leaves accuracy exactly unchanged). No routing prior improves raw calibration – including the accuracy-winning learned graph – but temperature scaling sharply reduces NLL for every configuration at no accuracy cost.

6 Conclusion

We presented SMART, a recursive marker-guided transformer whose central novelty is a *biology-informed router* that folds a gene-gene network-centrality prior into a Mixture-of-Recursions depth decision, so biology shapes where the model spends computation rather than only how its results are read. Across 11 single-cell and multi-omics datasets we find that a *learned* routing graph is the decisive positive (single-cell macro-F1 70.8%, +11.7 over a no-prior router), while a *fixed* hand-built biology prior does not separate from a random-graph control. By learning marker genes, compressing around them, and sharing one block across recursive refinement, SMART classifies with several times fewer transformer parameters than independent layers, a $\sim 38\%$ compute saving, and an interpretable compute-allocated recursion-depth signal. The complete pipeline, including all experiments and this paper, regenerates from a single command.

7 Broader Impact and Ethics Statement

SMART targets cell-type annotation and bulk-omics subtyping with far fewer parameters and an auditable marker-gene and recursion-depth signal, lowering the barrier for interpretable biological discovery. Its predictions are research tools, not clinical decisions: on a tissue or population absent from training they can be confidently wrong (our hard suites, e.g. Segerstolpe and STAD), so we report per-dataset error bars and degree-matched controls rather than a single headline number, and the learned marker panels should be inspected for confounds (batch, donor, ambient RNA) before any biological conclusion. All datasets are public and de-identified, used under their original licenses; extensions to non-public cohorts should pass the corresponding IRB and

data-governance review. Every run fits on a single GPU, and the full pipeline regenerates from one command with all reported numbers tracing to committed result files; automated tooling assisted code and manuscript preparation.

References

- Aran, D.; Looney, A. P.; Liu, L.; Wu, E.; Fong, V.; Hsu, A.; et al. 2019. Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage. *Nature Immunology* 20:163–172.
- Bae, S.; Kim, Y.; Bayat, R.; Kim, S.; Ha, J.; et al. 2025. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Balin, M. F.; Abid, A.; and Zou, J. 2019. Concrete autoencoders: Differentiable feature selection and reconstruction. In *International Conference on Machine Learning (ICML)*.
- BMFM-RNA authors. 2025. Bmfm-rna: A foundation-model pipeline with whole-cell expression denoising objectives. (*recent work; citation to be finalized*).
- Choromanski, K.; Likhoshesterov, V.; Dohan, D.; Song, X.; Gane, A.; Sarlos, T.; et al. 2021. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*.
- Cortal, A.; Martignetti, L.; Six, E.; and Rausell, A. 2021. Gene signature extraction and cell identity recognition at the single-cell level with cell-id. *Nature Biotechnology* 39:1095–1102.
- Cui, H.; Wang, C.; Maan, H.; Pang, K.; Luo, F.; Duan, N.; and Wang, B. 2024. scgpt: Toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods* 21:1470–1480.
- Dehghani, M.; Gouws, S.; Vinyals, O.; Uszkoreit, J.; and Kaiser, L. 2019. Universal transformers. In *International Conference on Learning Representations (ICLR)*.
- DOGMA authors. 2024. Dogma: Deterministic ontology- and phylogeny-guided topology for single-cell models. (*recent work; citation to be finalized*).
- Elmarakeby, H. A.; Hwang, J.; Arafah, R.; et al. 2021. Biologically informed deep neural network for prostate cancer discovery. *Nature* 598:348–352.
- Franzén, O.; Gan, L.-M.; and Björkegren, J. L. 2019. PanglaoDB: A web server for exploration of mouse and human single-cell RNA sequencing data. *Database* 2019:baz046.
- GeneMamba authors. 2024. Genemamba: Linear-time state-space models for single-cell transcriptomics. (*recent work; citation to be finalized*).
- Gillespie, M.; Jassal, B.; Stephan, R.; Milacic, M.; Rothfels, K.; Senff-Ribeiro, A.; Griss, J.; et al. 2022. The reactome pathway knowledgebase 2022. *Nucleic Acids Research* 50(D1):D687–D692.
- Graves, A. 2016. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.
- Guo, C.; Pleiss, G.; Sun, Y.; and Weinberger, K. Q. 2017. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*.

Hao, M.; Gong, J.; Zeng, X.; Liu, C.; Guo, Y.; Cheng, X.; Wang, T.; Ma, J.; Zhang, X.; and Song, L. 2024. Large-scale foundation model on single-cell transcriptomics. *Nature Methods* 21:1481–1491.

Howlader, K.; Islam, M. T.; and Le, W. 2026. Graph transformer-based pathway embedding for cancer prognosis. *arXiv preprint arXiv:2604.16685*.

Hu, C.; Li, T.; Xu, Y.; Zhang, X.; Li, F.; Bai, J.; Chen, J.; Jiang, W.; Yang, K.; Ou, Q.; Li, X.; Wang, P.; and Zhang, Y. 2023. Cellmarker 2.0: An updated database of manually curated cell markers in human/mouse and web tools based on scRNA-seq data. *Nucleic Acids Research* 51(D1):D870–D876.

Ianevski, A.; Giri, A. K.; and Aittokallio, T. 2022. Fully-automated and ultra-fast cell-type identification using specific marker combinations from single-cell transcriptomic data. *Nature Communications* 13:1246.

Islam, M. T., and Xing, L. 2023. Cartography of genomic interactions enables deep analysis of single-cell expression data. *Nature Communications* 14:679.

Jaber, J., and Jaber, O. 2026. Ouroboros: Dynamic weight generation for recursive transformers via input-conditioned LoRA modulation. *arXiv preprint*.

Jang, E.; Gu, S.; and Poole, B. 2017. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations (ICLR)*.

Lan, Z.; Chen, M.; Goodman, S.; Gimpel, K.; Sharma, P.; and Soricut, R. 2020. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*.

Levine, D.; Rizvi, S. A.; Lévy, S.; Pallikkavaliyaveetil, N.; and van Dijk, D. 2024. Cell2sentence: Teaching large language models the language of biology. In *International Conference on Machine Learning (ICML)*.

Lopez, R.; Regier, J.; Cole, M. B.; Jordan, M. I.; and Yosef, N. 2018. Deep generative modeling for single-cell transcriptomics. *Nature Methods* 15:1053–1058.

Luecken, M. D., and Theis, F. J. 2019. Current best practices in single-cell RNA-seq analysis: A tutorial. *Molecular Systems Biology* 15(6):e8746.

Raposo, D.; Ritter, S.; Richards, B.; Lillicrap, T.; Humphreys, P. C.; and Santoro, A. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*.

Regev, A.; Teichmann, S. A.; Lander, E. S.; Amit, I.; Benoist, C.; Birney, E.; et al. 2017. The human cell atlas. *eLife* 6:e27041.

scTransformer authors. 2024. sctransformer: Prior-gated attention with transcription-factor regulatory masks for single-cell modeling. (*recent work; citation to be finalized*).

Shazeer, N.; Mirhoseini, A.; Maziarz, K.; Davis, A.; Le, Q.; Hinton, G.; and Dean, J. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*.

Shen, Z.; Liu, Z.; and Xing, E. P. 2022. Sliced recursive transformer. In *European Conference on Computer Vision (ECCV)*.

Souza, and Mehta. 2024. Simple linear baselines rival transformer foundation models on single-cell cell-type annotation. (*recent work; citation to be finalized*).

The Tabula Sapiens Consortium. 2022. The tabula sapiens: A multiple-organ, single-cell transcriptomic atlas of humans. *Science* 376(6594):eabl4896.

Theodoris, C. V.; Xiao, L.; Chopra, A.; Chaffin, M. D.; Al Sayed, Z. R.; Hill, M. C.; Mantineo, H.; Brydon, E. M.; Zeng, Z.; Liu, X. S.; and Ellinor, P. T. 2023. Transfer learning enables predictions in network biology. *Nature* 618:616–624.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Wang, S.; Li, B. Z.; Khabsa, M.; Fang, H.; and Ma, H. 2020. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*.

Wen, H.; Tang, W.; Dai, X.; Ding, J.; Jin, W.; Xie, Y.; and Tang, J. 2024. CellPLM: Pre-training of cell language model beyond single cells. In *International Conference on Learning Representations (ICLR)*.

Wolf, F. A.; Angerer, P.; and Theis, F. J. 2018. SCANPY: Large-scale single-cell gene expression data analysis. *Genome Biology* 19:15.

Xiong, Y.; Zeng, Z.; Chakraborty, R.; Tan, M.; Fung, G.; Li, Y.; and Singh, V. 2021. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *AAAI Conference on Artificial Intelligence*.

Yang, F.; Wang, W.; Wang, F.; Fang, Y.; Tang, D.; Huang, J.; Lu, H.; and Yao, J. 2022. scbert as a large-scale pretrained deep language model for cell type annotation of single-cell RNA-seq data. *Nature Machine Intelligence* 4:852–866.

A Dataset Details

We use 8 genomap single-cell datasets (Islam and Xing 2023) converted to plain CSV (expression + labels + stratified split) – Baron, Lung, Muraro, Oesophagus, Segerstolpe, Spleen, T-cell and Xin – and 3 Reactome/P-NET multi-omics cohorts (prostate, BLCA, STAD) whose fixed Reactome pathway tokens pool mutation, copy-number and expression channels. Per-dataset sample, feature and class counts are read directly from the result files and summarised in Table 12.

B Effective-FLOPs Accounting

The compute numbers report per-sample FLOPs of the recursive transformer stack, the only component routing changes. One application of the shared block to a tokens costs $\phi(a) = 4a^2d + 4ad d_{\text{ff}}$; the nominal fixed-depth cost is $\Phi_{\text{nom}} = K \phi(M)$ and the effective cost sums one block over the tokens active at each step, $\Phi_{\text{eff}} = \sum_{t=1}^K \phi(a_t)$, where a_t is the mean number of markers the expert-choice funnel keeps at step t . Gene embedding and marker selection are

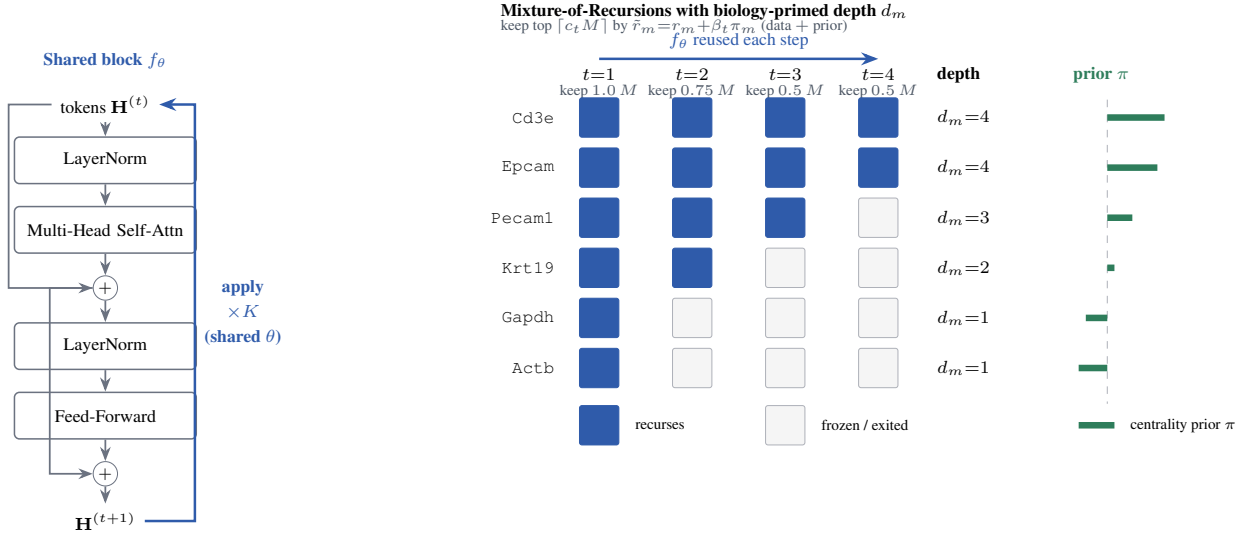


Figure 2: **Biology-informed Mixture-of-Recursions.** *Left:* one weight-shared pre-norm block f_θ (the model’s only transformer parameters) is re-applied up to $K=4$ times, so depth costs no extra parameters. *Right:* an expert-choice router keeps a shrinking top fraction of markers per step (capacity funnel $1, \frac{3}{4}, \frac{1}{2}, \frac{1}{2}$); a marker not kept is frozen, so its *recursion depth* d_m is the deepest step it survived. The keep decision adds a label-free genomap co-expression-centrality prior $\beta_t \pi_m$ to each logit (Eq. 1); the **green bars** show that prior π (longer = more central a hub), so lineage and hub genes (Cd3e, Epcam) are primed to recur deepest while settled house-keeping genes (Gapdh, Actb) exit at $d_m=1$. The bar heights are illustrative of the centrality ordering, not fitted values. The prior graph may be *fixed* (shown) or *learned* end-to-end; empirically the learned graph is the decisive win, while a fixed centrality prior does not beat a random-graph control.

Dataset	Modality	Samples	Features	Classes
Baron	single-cell	8,569	2,000	13
Lung	single-cell	57,020	1,089	25
Muraro	single-cell	2,122	2,000	9
Oeso.	single-cell	87,947	1,089	18
Seger.	single-cell	2,127	2,000	11
Spleen	single-cell	94,129	1,089	28
T-cell	single-cell	24,007	1,089	7
Xin	single-cell	1,492	2,000	4
Prostate	multi-omics	1,011	1,223	2
BLCA	multi-omics	404	1,258	2
STAD	multi-omics	414	1,258	2

Table 12: The 11 datasets used throughout: 8 genomap single-cell suites and 3 Reactome/P-NET multi-omics cohorts. Counts are read directly from the result files.

$\mathcal{O}(Nd)$, identical across routing modes, and excluded from this stack-level comparison.

End-to-end accounting. For completeness we account for the whole forward pass, not only the stack. Three parts scale with the full gene count N : gene embedding $\Theta(Nd)$; the cross-attention marker router, whose M queries attend over all N gene keys at $\Theta(MNd)$; and the optional gene-graph smoother, which at rank r costs $\Theta(Nr)$ (never the N^2 dense form). Everything after marker selection – the

recursive stack, pooling and head – scales with the compressed budget $M \ll N$ at $\Theta(M^2d)$ per pass. So the router’s $\Theta(MNd)$ term dominates the N -scaling and is *linear* in N (versus the $\Theta(N^2d)$ self-attention a full-gene transformer would pay), while the quadratic cost is paid only on the M markers. The architecture ablations of Table 7 vary only the stack, so the stack-level ratios there are the correct comparison for the routing/sharing claims; the router and embedding terms are shared by every variant. Measured wall-clock and peak-memory profiling on matched hardware is a straightforward addition we leave to the camera-ready.

C Theoretical Foundation of the Router

This appendix gives the mathematical and biological grounding for SMART’s biology-informed router.

Routing as conditional computation. The router implements *conditional computation*: a learned policy routes each token to a token-specific amount of compute, the gating principle of sparsely-gated mixtures of experts (Shazeer et al. 2017), reused by Mixture-of-Depths (Raposo et al. 2024) and Mixture-of-Recursions (Bae et al. 2025); the “experts” here are recursion *depths* of one shared block, which couples adaptive computation (Graves 2016) to weight sharing.

The differentiable handle. The discrete top- k has zero gradient almost everywhere, so SMART keeps the soft probability of the chosen route as a multiplicative *gate* on the block

output, $\mathbf{o}_m = g_m f_\theta(\mathbf{h}_m)$ with $g_m = \sigma(\tilde{r}_m)$. Because g_m is smooth in $\mathbf{w}_r$, the chain $\mathcal{L} \leftarrow \mathbf{o}_m \leftarrow g_m \leftarrow \mathbf{w}_r$ is unbroken: the hard choice routes, the soft gate carries the gradient. The biological prior of Eq. (1) is an additive constant in this logit, so it shifts the decision without breaking this path.

Biological foundation of the prior. Two facts from single-cell biology motivate the prior. A small set of *marker* genes carries most discriminative signal (Ianevski, Giri, and Aittokallio 2022; Franzén, Gan, and Björkegren 2019; Hu et al. 2023), so compute should be allocated unevenly. And gene co-expression networks are approximately scale-free: a few high-degree *hub* genes (master regulators) exert outsized influence. We operationalise “hub” as eigenvector centrality on the genomap co-expression graph $\mathbf{W}$: the leading eigenvector $\mathbf{v}$ of $\mathbf{W}\mathbf{v} = \lambda\mathbf{v}$ scores each gene by the centrality of its neighbours, recursively, and we z -score it to form π .

Empirical-Bayes reading. Equation (1) is a log-linear prior on the routing decision, with $\beta_t \pi_m$ a Gaussian-like prior mean and $\beta_t = \beta_0(1 - \text{progress})$ a shrinkage strength that decays as data evidence accumulates: prior-dominated when the likelihood is uninformative (early training), data-dominated later.

Leakage safety. $\mathbf{W}$ is computed from training-split *expression only*; no cell-type label enters it. The prior therefore injects network topology, not the answer, which is why a gene-discovery claim under this prior is not circular, unlike a prior built from curated cell-type marker lists.

Optional pathway-graph smoothing. The additive bias treats genes independently. Co-pathway genes should route coherently, which one obtains by smoothing the logits over $\mathbf{W}$ with the normalised Laplacian $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{W}\mathbf{D}^{-1/2}$, $\hat{\mathbf{r}}^{(t)} = \tilde{\mathbf{r}}^{(t)} - \gamma \mathbf{L}\tilde{\mathbf{r}}^{(t)}$ (graph-Laplacian regularisation): a gene borrows routing strength from its network neighbours. This is one sparse matrix-vector product per step and adds no transformer parameters; we expose it as an option and leave its evaluation to future work.